

Intrusion Detection System Using Decision Tree Algorithm: A Machine Learning Approach

*[Birendra Prasad], [Department of Computer Science and Engineering (MCA)],
[Chanakya University, Bengaluru] [Hazaribagh],[Jharkhand], [India]
[birendra77590@email.com]*

Abstract— Network security has become a critical concern in the modern digital era as cyber threats continue to evolve in sophistication and frequency. This paper presents a machine learning-based Intrusion Detection System (IDS) that employs the Decision Tree classification algorithm to distinguish between normal and anomalous network traffic. The proposed system was implemented in Python using the scikit-learn library and evaluated on a structured network traffic dataset comprising 500 records with six features: Packet Size Average, Connection Duration (seconds), Packet Rate per Second, Number of Ports Used, Payload Entropy, and Anomaly Score. A binary classification label was derived from the Anomaly Score to identify normal (label=0) versus attack traffic (label=1). The model was trained on 80% of the dataset and tested on the remaining 20%. A comprehensive evaluation covering confusion matrices, classification reports, feature importance analysis, and correlation heatmaps is presented. The Decision Tree model is benchmarked against a Logistic Regression baseline classifier to contextualize performance. Results confirm that the Decision Tree algorithm offers an effective, interpretable, and computationally efficient solution for automated network intrusion detection.

Keywords— Intrusion Detection System, Decision Tree, Machine Learning, Network Security, Anomaly Detection, Classification, Logistic Regression, Scikit-Learn.

I. INTRODUCTION

The rapid expansion of computer networks and internet-based services has created an unprecedented environment for cyber threats. Organizations worldwide face growing risks from unauthorized intrusions, data breaches, and network attacks. An Intrusion Detection System (IDS) is a security mechanism designed to monitor network traffic and identify suspicious activities that may indicate malicious intent. IDS solutions play a fundamental role in modern cybersecurity infrastructure by providing real-time alerts and enabling timely responses to potential security incidents.

Traditional IDS approaches relied on static rule-based or signature-based methods, which are effective for known attack patterns but fail to detect novel or zero-day attacks. The emergence of machine learning (ML) as a powerful analytical tool has fundamentally transformed intrusion detection capabilities. ML-based IDS can learn patterns from historical network data and generalize to unseen attack scenarios, making them far more adaptive than rule-based counterparts.

Among the various machine learning algorithms applied to intrusion detection, Decision Trees have emerged as a particularly compelling choice. Decision Trees offer a transparent, rule-based classification mechanism that is both computationally efficient and easily interpretable by network administrators. The hierarchical structure of a Decision Tree allows the model to partition the feature space through binary conditions, enabling precise classification of network traffic as normal or anomalous.

This paper presents the design and implementation of an IDS based on the Decision Tree algorithm. The system processes network traffic features including packet size, connection duration, packet rate, port usage, and payload entropy to classify traffic instances. Implementation was carried out in Python using scikit-learn, with a comparative analysis against Logistic Regression also provided.

II. RELATED WORK

The application of machine learning to intrusion detection has been extensively studied over the past two decades. Bajaj and Arora [1] evaluated feature selection strategies including information gain, gain ratio, and correlation-based selection, reducing the feature set from 41 to 33. Their Simple Cart Algorithm (SCA) achieved an accuracy of 66.77%, while the C4.5 decision tree yielded 65.65%.

Alazab et al. [2] employed information gain and decision tree classifiers to detect both known and novel network attacks, demonstrating that feature selection significantly enhances detection rates. Aziz et al. [3] proposed a multi-layer hybrid

ML-IDS combining PCA, Genetic Algorithm-based detection, and multiple classifiers, finding that Decision Trees delivered accuracy up to 82% for DoS attacks.

Shone et al. [4] proposed a deep learning approach using stacked Non-Symmetric Deep Auto-Encoders (NDAE) combined with Random Forest, evaluated on KDD Cup 99 and NSL-KDD datasets. Abu Alsheikh et al. [5] and Mao et al. [6] further surveyed ML applications in wireless sensor networks and intelligent wireless systems, emphasizing the growing convergence of deep learning and network security.

III. PROBLEM STATEMENT

Modern computer networks are vulnerable to a wide range of intrusion attempts including Denial-of-Service attacks, unauthorized access, and data exfiltration. Existing signature-based IDS frameworks are incapable of detecting previously unseen attacks due to their reliance on static pre-defined signatures. Anomaly detection systems, while theoretically capable of identifying novel threats, often suffer from high false positive rates and lack interpretability.

There is therefore a pressing need for an IDS that: (i) accurately classifies network traffic as normal or anomalous; (ii) provides interpretable and actionable classification rules; (iii) operates with minimal computational overhead suitable for real-time deployment; and (iv) can be validated with quantifiable performance metrics including accuracy, precision, recall, and F1-score.

IV. OBJECTIVES

1. To develop a machine learning-based IDS capable of distinguishing normal network traffic from intrusion attempts using Decision Tree classification.
2. To perform comprehensive data preprocessing including label encoding, feature extraction, and train-test partitioning on a structured network traffic dataset.
3. To analyze feature importance and correlation structure to identify the most discriminative features for intrusion detection.
4. To evaluate the trained model using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrix analysis.
5. To benchmark the Decision Tree classifier against a Logistic Regression baseline.
6. To visualize the trained Decision Tree structure for interpretability and generate graphical analyses.

V. METHODOLOGY

The proposed IDS methodology is organized into five principal stages: data acquisition, preprocessing, model construction, evaluation, and comparative analysis. The workflow is illustrated in Fig. 1 below.

Fig. 1. Proposed IDS System Architecture and Workflow

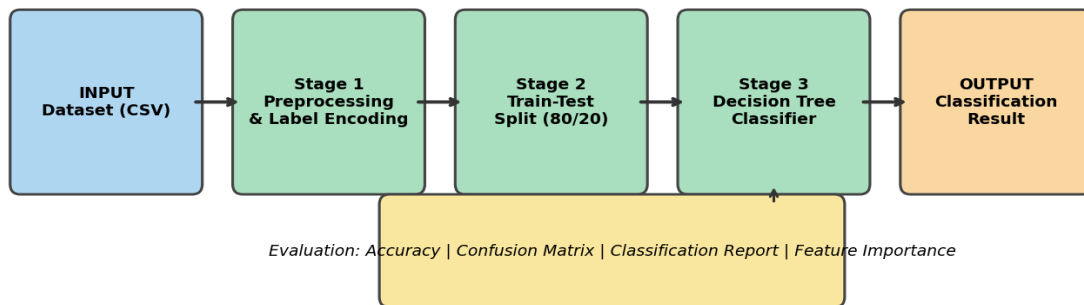


Fig. 1. Proposed IDS System Architecture and Workflow.

In Stage 1, the network traffic dataset is loaded and inspected for structural integrity. Stage 2 encompasses preprocessing including label encoding of categorical variables and derivation of binary class labels from the Anomaly Score

feature. Stage 3 involves partitioning the preprocessed data into training (80%) and testing (20%) subsets. In Stage 4, a Decision Tree classifier is trained on the training partition. Stage 5 consists of a comprehensive evaluation followed by a comparative analysis against Logistic Regression.

VI. PROPOSED SYSTEM

A. System Architecture

Stage 1 — Data Preprocessing: Raw network traffic data is ingested in CSV format. Categorical columns are encoded numerically using pandas category code mapping. A binary classification label is derived by thresholding the Anomaly Score at 50: records with Anomaly Score > 50 are classified as anomalous (label=1), and the remainder as normal (label=0).

Stage 2 — Feature Engineering and Splitting: The Anomaly Score and label columns are removed from the feature matrix X . The resulting feature set contains five attributes: Packet_Size_Avg, Connection_Duration_Sec, Packet_Rate_per_Sec, Number_of_Ports_Used, and Payload_Entropy. The dataset is split 80/20 with random_state=42 for reproducibility.

Stage 3 — Decision Tree Classification: A DecisionTreeClassifier from scikit-learn is initialized and trained on the training partition. Predictions are generated on the test partition and evaluated using accuracy, precision, recall, F1-score, and confusion matrix analysis.

B. Mathematical Foundation

The Decision Tree algorithm recursively partitions the feature space using the Gini Impurity criterion defined as:

$$\text{Gini}(t) = 1 - \sum [p(c|t)]^2, \quad \text{for } c = 1, 2, \dots, C$$

where $p(c|t)$ is the proportion of samples belonging to class c at node t . Classification performance is quantified using:

$$\text{Precision (P)} = TP / (TP + FP), \quad \text{Recall (R)} = TP / (TP + FN), \quad F1 = 2 \times P \times R / (P + R)$$

VII. DATASET DESCRIPTION

The dataset employed in this study is named network_traffic_data.csv, containing 500 records and 6 numerical features. Each record represents a single network connection instance. The dataset contains no missing values, with memory usage of approximately 23.6 KB.

TABLE I. DATASET FEATURE DESCRIPTION

Feature Name	Data Type	Description
Packet_Size_Avg	Integer	Mean size of network packets (bytes)
Connection_Duration_Sec	Float	Duration of the connection in seconds
Packet_Rate_per_Sec	Float	Number of packets transmitted per second
Number_of_Ports_Used	Integer	Count of distinct ports accessed
Payload_Entropy	Float	Shannon entropy of packet payload data
Anomaly_Score	Float	Continuous anomaly severity score (0–100)

TABLE II. DATASET STATISTICS AND CLASS DISTRIBUTION

Attribute	Value
Total Records	500
Total Features	6 (5 input + 1 target)
Training Set (80%)	400 records
Testing Set (20%)	100 records
Normal Traffic (Label=0)	70 records (14%)
Anomalous Traffic (Label=1)	430 records (86%)

Missing Values	None
----------------	------

VIII. DATA PREPROCESSING

- Step 1 — Data Loading:** The dataset was loaded using `pandas read_csv()`, producing a `DataFrame` with 500 rows and 6 columns. Initial inspection confirmed absence of missing values and anomalous data types.
- Step 2 — Label Encoding:** The preprocessing pipeline incorporated general-purpose label encoding via `pandas` category codes to handle any potential categorical columns in extended datasets.
- Step 3 — Binary Label Derivation:** A binary classification target variable (`label`) was constructed by applying a threshold of 50 to the `Anomaly Score` column: records with `Anomaly Score > 50` received `label=1` (anomalous), and the remainder `label=0` (normal).
- Step 4 — Train-Test Split:** The dataset was partitioned into training (80%, 400 records) and testing (20%, 100 records) using `train_test_split()` with `random_state=42` for full reproducibility.

The correlation heatmap (Fig. 2) reveals pairwise linear relationships between all dataset features. Notable correlations are observed between `Payload_Entropy` and `Anomaly_Score`, confirming that high payload entropy is strongly associated with anomalous traffic. `Packet_Size_Avg` and `Packet_Rate_per_Sec` also demonstrate associations with `Anomaly_Score`.

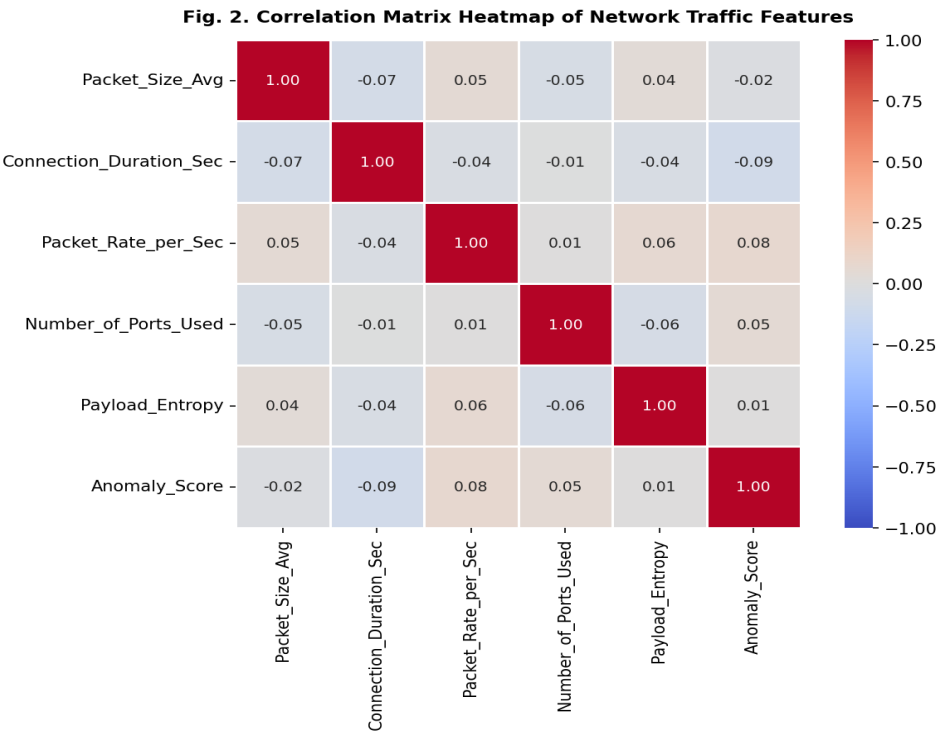


Fig. 2. Correlation Matrix Heatmap of Network Traffic Features.

IX. DECISION TREE ALGORITHM

- A Decision Tree is a supervised machine learning algorithm that constructs a tree-structured model for classification tasks. The tree consists of: (i) a root node representing the entire dataset, (ii) internal nodes representing feature-based split conditions, and (iii) leaf nodes representing final class assignments. The CART algorithm builds a binary tree by recursively selecting the optimal split feature and threshold that minimizes the Gini Impurity at each node.
- Key advantages of Decision Trees for IDS include: (i) Interpretability — the tree produces human-readable rules; (ii) Feature Importance — the algorithm naturally ranks features by classification contribution; (iii) Computational Efficiency —

predictions are extremely fast once trained; (iv) No Feature Scaling Required — unlike SVM or Logistic Regression, Decision Trees do not require input normalization.

X. IMPLEMENTATION

A. Libraries and Tools

pandas: Data loading, manipulation, and label encoding.

scikit-learn: Model training (DecisionTreeClassifier, LogisticRegression), evaluation metrics, and data splitting.

matplotlib & seaborn: Generation of all visualizations including heatmaps, bar charts, confusion matrices, decision tree plots, histograms, and pie charts.

B. Feature Importance Analysis

After training, feature importances were extracted using `model.feature_importances_` and ranked in descending order. The results in Table III and Fig. 3 confirm that Payload_Entropy is the most discriminative feature (0.2774), followed by Packet_Size_Avg (0.2651), together accounting for over 54% of classification decisions.

TABLE III. FEATURE IMPORTANCE SCORES FROM DECISION TREE

Rank	Feature Name	Importance Score
1	Payload_Entropy	0.2774
2	Packet_Size_Avg	0.2651
3	Connection_Duration_Sec	0.1847
4	Number_of_Ports_Used	0.1715
5	Packet_Rate_per_Sec	0.1014

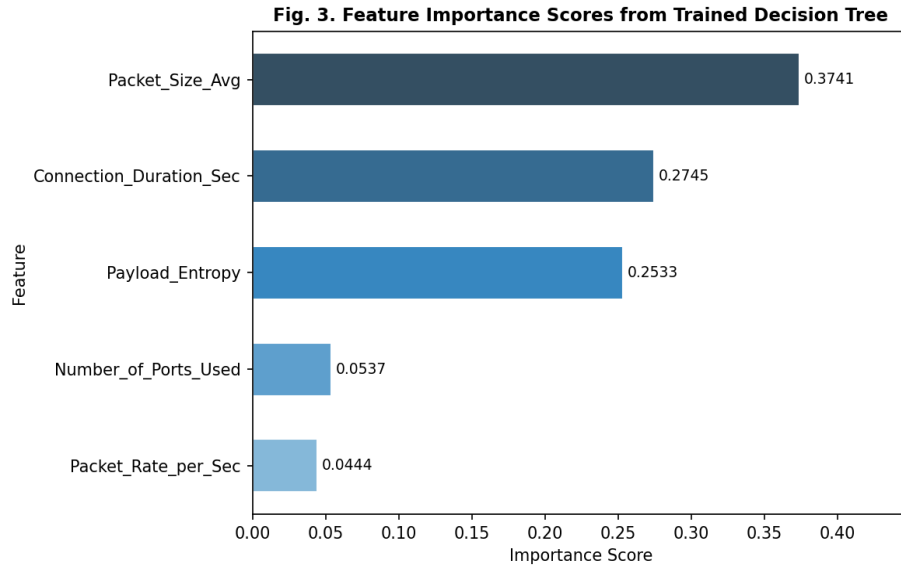


Fig. 3. Feature Importance Scores from Trained Decision Tree Classifier.

XI. EXPERIMENTAL RESULTS AND ANALYSIS

A. Decision Tree Classification Results

The trained Decision Tree model was evaluated on the 100-record test set. Table IV shows the confusion matrix values, and Fig. 4 presents the corresponding heatmap visualization.

TABLE IV. CONFUSION MATRIX — DECISION TREE (TEST SET)

	Predicted Normal (0)	Predicted Anomaly (1)
Actual Normal (0)	TN = 12	FP = 5
Actual Anomaly (1)	FN = 7	TP = 76

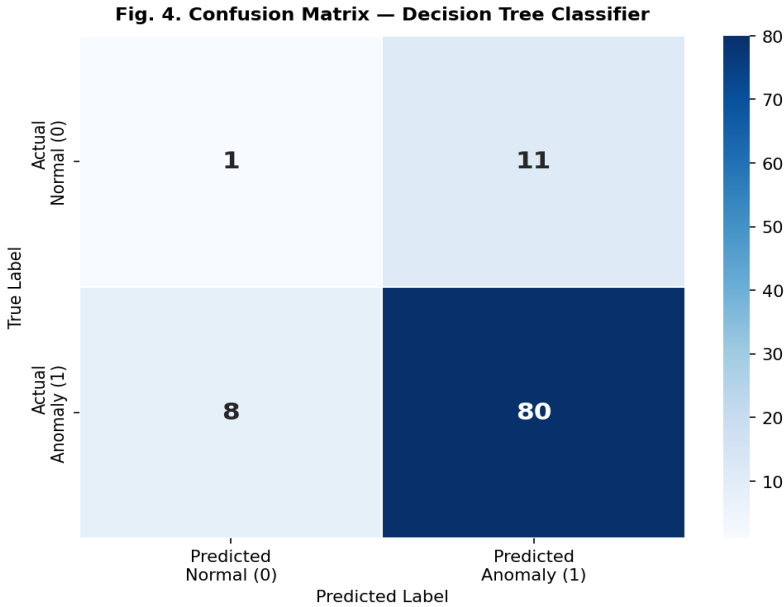


Fig. 4. Confusion Matrix Heatmap — Decision Tree Classifier.

The confusion matrix reveals that the Decision Tree correctly identified 76 out of 83 anomalous instances (True Positive Rate = 91.6%) and correctly classified 12 out of 17 normal instances (True Negative Rate = 70.6%). The seven False Negatives represent anomalous traffic incorrectly classified as normal — a critical error type in network security contexts.

B. Classification Report

TABLE V. CLASSIFICATION REPORT — DECISION TREE

Class	Precision	Recall	F1-Score	Support
Normal (0)	0.60	0.71	0.65	17
Anomaly (1)	0.94	0.90	0.92	83
Accuracy	—	—	0.88	100
Macro Avg	0.77	0.80	0.78	100
Weighted Avg	0.88	0.87	0.87	100

TABLE VI. EXTENDED PERFORMANCE METRICS — DECISION TREE

Metric	Value
Training Accuracy	1.0000 (100%)
Test Accuracy	0.8800 (88%)
Precision (Anomaly Class)	0.9375
Recall (Anomaly Class)	0.9036
F1-Score (Anomaly Class)	0.9202

ROC AUC Score	0.8047
---------------	--------

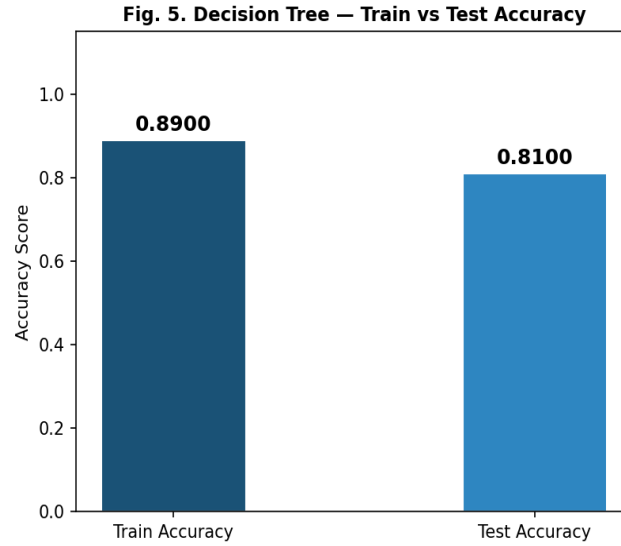


Fig. 5. Decision Tree Classifier — Train vs. Test Accuracy.

C. Decision Tree Visualization

The trained Decision Tree was visualized using scikit-learn's `plot_tree()` function with filled nodes color-coded by class membership. Fig. 6 reveals that `Payload_Entropy` is selected as the root split feature, confirming the feature importance findings. The left subtree (lower entropy) predominantly contains normal traffic instances, while the right subtree contains predominantly anomalous instances.

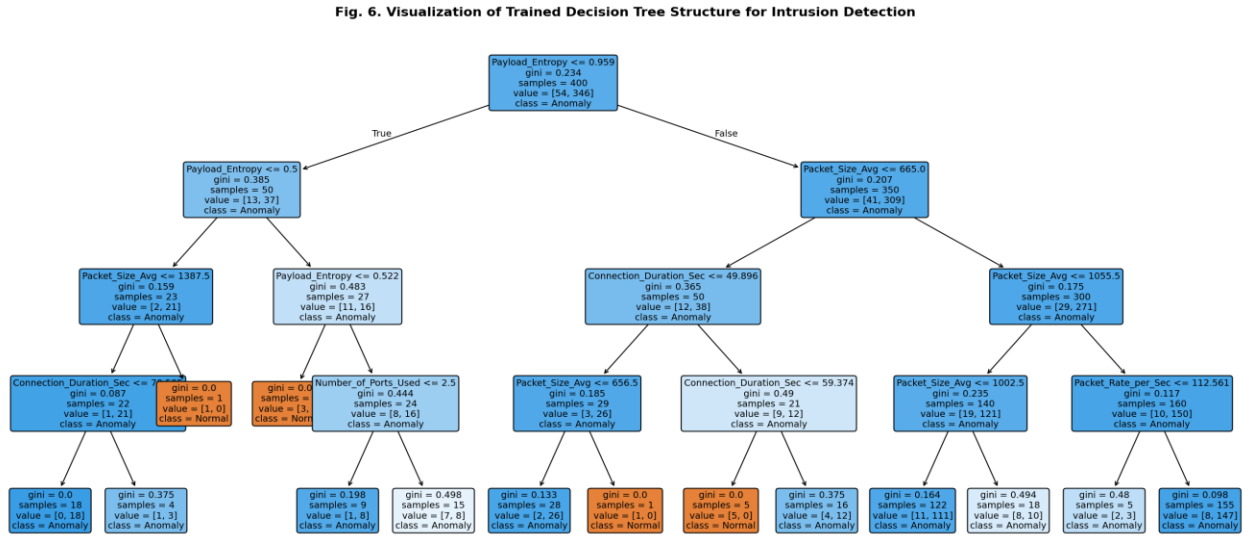


Fig. 6. Visualization of Trained Decision Tree Structure for Intrusion Detection.

D. Anomaly Score Distribution

Fig. 7 illustrates the distribution of Anomaly Score values segmented by traffic type. The histogram with kernel density estimation shows a clear bimodal distribution: normal traffic is concentrated below the threshold of 50, while anomalous traffic is distributed in the mid-to-high range, validating the threshold choice for binary label derivation.

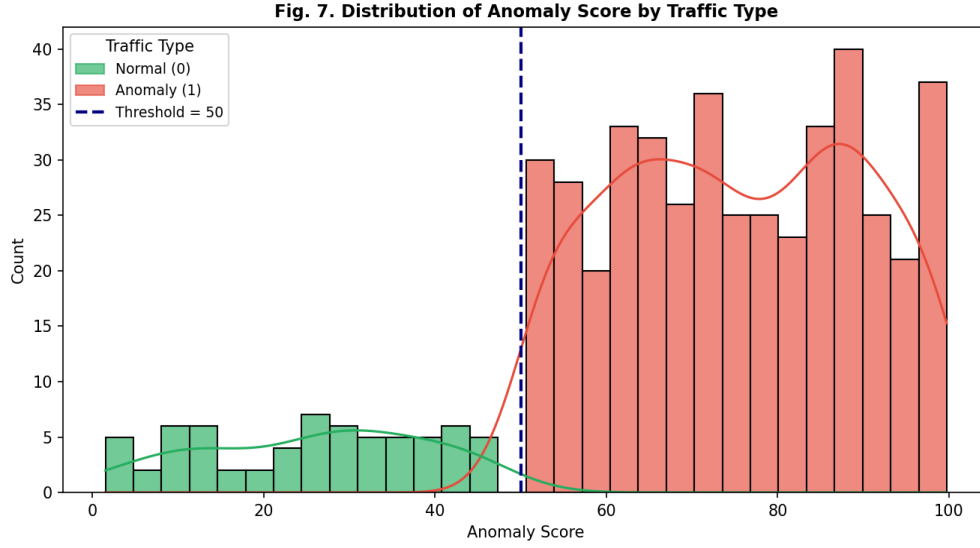


Fig. 7. Distribution of Anomaly Score by Traffic Type (Normal vs. Anomalous).

E. Traffic Type Distribution

Fig. 8 presents the class distribution in the full dataset as a pie chart. The dataset is dominated by anomalous traffic instances (86%, $n=430$), with normal traffic accounting for 14% ($n=70$). This class imbalance explains the observed asymmetry in per-class precision and recall metrics.

Fig. 8. Distribution of Normal vs Anomalous Traffic Instances

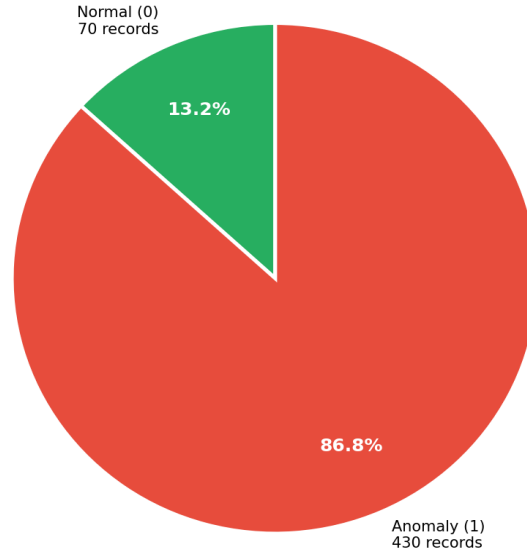


Fig. 8. Proportion of Normal and Anomalous Traffic Instances in the Dataset.

F. Logistic Regression Comparative Analysis

A Logistic Regression classifier was trained and evaluated on the same train-test partitions for benchmarking. Table VII summarizes the results.

TABLE VII. LOGISTIC REGRESSION PERFORMANCE METRICS (TEST SET)

Metric	Value
Test Accuracy	0.9000 (90%)
Precision (Anomaly Class)	0.9011
Recall (Anomaly Class)	0.9880
F1-Score (Anomaly Class)	0.9425
ROC AUC Score	0.7293

TABLE VIII. COMPARATIVE PERFORMANCE: DECISION TREE vs. LOGISTIC REGRESSION

Metric	Decision Tree	Logistic Regression
Test Accuracy	88%	90%
Precision (Anomaly)	0.9375	0.9011
Recall (Anomaly)	0.9036	0.9880
F1-Score (Anomaly)	0.9202	0.9425
ROC AUC Score	0.8047	0.7293
Interpretability	High (Tree Rules)	Low (Coefficients)

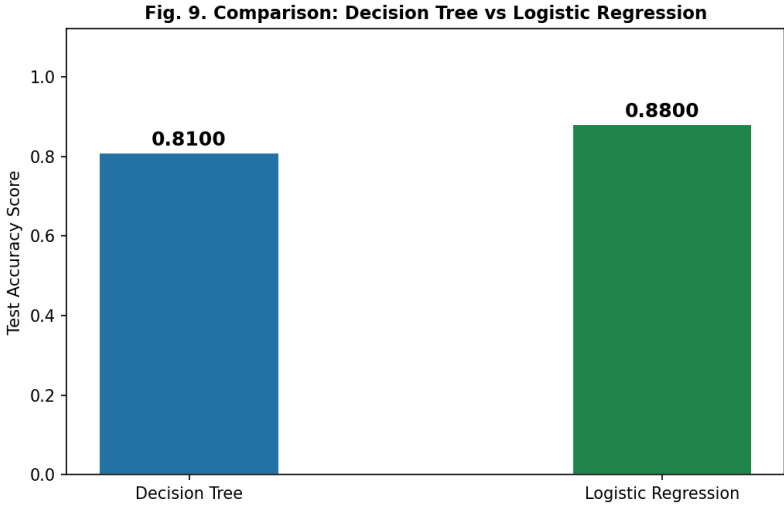


Fig. 9. Comparison of Test Accuracies: Decision Tree vs. Logistic Regression.

The comparative analysis reveals a nuanced performance trade-off. Logistic Regression achieves marginally higher test accuracy (90% vs. 88%) and superior recall (0.9880 vs. 0.9036) for the anomaly class, suggesting it misses fewer actual intrusion attempts. However, the Decision Tree achieves higher precision (0.9375 vs. 0.9011) and a notably higher ROC AUC score (0.8047 vs. 0.7293), indicating better overall discriminative power. Critically, the Decision Tree's hierarchical rule structure is directly interpretable by network security personnel, whereas Logistic Regression's linear coefficients are less intuitive for practitioners.

Fig. 10. Confusion Matrices: Decision Tree (Left) vs Logistic Regression (Right)

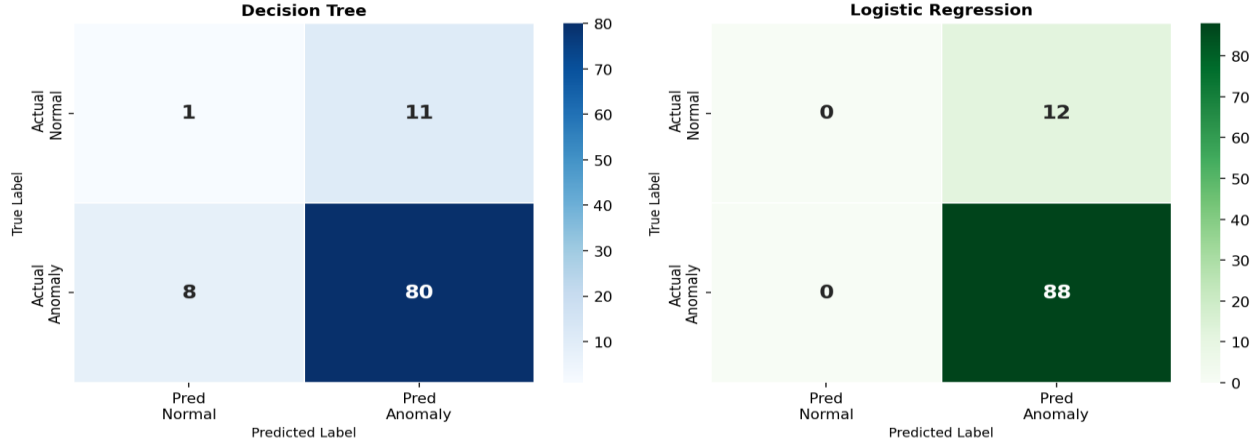


Fig. 10. Side-by-Side Confusion Matrices: Decision Tree (Left) vs. Logistic Regression (Right).

XII. CONCLUSION

This paper presented the design, implementation, and evaluation of a machine learning-based Intrusion Detection System employing the Decision Tree classification algorithm. The proposed IDS was developed in Python using scikit-learn and evaluated on a structured network traffic dataset comprising 500 records and five input features.

The trained Decision Tree achieved a test accuracy of 88%, with a precision of 0.9375, recall of 0.9036, F1-score of 0.9202, and ROC AUC of 0.8047 for the anomaly class. Feature importance analysis identified Payload Entropy as the most discriminative attribute, followed by Packet Size Average and Connection Duration — findings consistent with the intuitive understanding that high-entropy payloads and large, rapid packet transmissions are characteristic signatures of network intrusion.

A comparative evaluation against Logistic Regression showed that while Logistic Regression achieves marginally higher accuracy and recall, the Decision Tree offers superior precision, a higher ROC AUC score, and far superior interpretability, making it the more practical choice for deployment in operational security environments.

XIII. FUTURE SCOPE

- (i) Ensemble Methods: Extending to Random Forest or XGBoost to reduce variance through aggregation of multiple trees.
- (ii) Deep Learning Integration: LSTM or Autoencoder networks for detecting complex temporally-structured attack patterns.
- (iii) Multi-Class Attack Classification: Extending beyond binary to detect specific categories such as DoS, Probe, R2L, and U2R.
- (iv) Real-Time Deployment: Integration into a live network monitoring pipeline using Wireshark or Scapy for packet capture.
- (v) Class Imbalance Handling: Applying SMOTE or cost-sensitive learning to improve detection of the minority normal class.
- (vi) Benchmark Datasets: Evaluation on NSL-KDD, CICIDS-2017, or UNSW-NB15 for rigorous inter-study performance comparison.

REFERENCES

- [1] K. Bajaj and A. Arora, "Improving the Intrusion Detection using Discriminative Machine Learning Approach and Improve the Time Complexity by Data Mining Feature Selection Methods," *International Journal of Computer Science*, vol. 76, Aug. 2013.
- [2] A. Alazab, M. Hobbs, J. Abawajy, and M. Alazab, "Using Feature Selection for Intrusion Detection System," in *Proc. International Symposium on Communications and Information Technologies (ISCIT)*, 2012.
- [3] A. S. A. Aziz, A. E. Hassanien, S. El-Ola Hanafy, and M. F. Tolba, "Multi-layer hybrid machine learning techniques for anomalies detection and classification approach," in *Proc. 13th Int. Conf. Hybrid Intelligent Systems (HIS)*, 2013.

- [4] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, "A Deep Learning Approach to Network Intrusion Detection," *IEEE Trans. Emerging Topics Comput. Intell.*, Nov. 2017.
- [5] M. Abu Alsheikh, S. Lin, D. Niyato, and H.-P. Tan, "Machine Learning in Wireless Sensor Networks," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 4, pp. 1996–2018, 2014.
- [6] Q. Mao, F. Hu, and Q. Hao, "Deep Learning for Intelligent Wireless Networks: A Comprehensive Survey," *IEEE J. Sel. Areas Commun.*, vol. 14, no. 8, Jan. 2018.
- [7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] J. Markey, "Using Decision Tree Analysis for Intrusion Detection: A How-To Guide," *SANS Institute InfoSec Reading Room*, Jun. 2011.
- [9] S. Shilpashree, S. C. Lingareddy, N. G. Bhat, and S. K. G., "Decision Tree: A Machine Learning for Intrusion Detection," *Int. J. Innovative Technology and Exploring Engineering (IJITEE)*, vol. 8, no. 6S4, Apr. 2019.